技术部第二次技术分享会

Java 基础语法

姚雨森　技术部分管主席

2026-10-23

让程序主动“开口”

我们需要知道程序正在执行什么，可以让它把信息打印到控制台。

使用 System.out.println(...) 输出一行文字：

```
System.out.println("Hello, Java!");
```

双引号里面的文字会被打印出来，双引号本身不会显示。

控制台输出：

```
Hello, Java!
```

把输出语句写在 main 中

程序从 main 方法开始执行。先把输出语句写在它的大括号内：

```
public class Main {
    public static void main(String[] args) {
        System.out.println("First");
        System.out.println("Second");
    }
}
```

这里的两条语句**从上到下依次执行**，每次输出后换行：

```
First
Second
```

注释：给人看的说明

注释用于解释代码，注释中的内容不会被当作语句执行。

- 单行注释：`//` 后面直到本行末尾的内容。
- 多行注释：`/*` 与 `*/` 之间的内容，可以跨行。

下面的代码写在 `main` 中：

```
// Print a greeting.
System.out.println("Hello!"); // This is a comment.
/* The next statement is commented out.
System.out.println("Hidden");
*/
System.out.println("Goodbye!");
```

控制台依次输出 `Hello!` 和 `Goodbye!`，各占一行；`Hidden` 不会输出。

变量：保存程序需要的数据

程序有时需要做运算，也需要保存一些数据，比如数值、文本。

可以用**变量**保存这些数据，再通过变量名读取或修改它们。

下面的代码写在 main 中：

```
int score = 60;
System.out.println(score);

score = 70;
System.out.println(score);
```

- `int score = 60;`：声明整数变量 score，初始化为 60。
- `score = 70;`：重新赋值，把 score 中保存的值改为 70。
- 两次 println 依次输出 60 和 70，各占一行。

变量的声明与赋值

声明变量时赋值（初始化）

语法：　类型　变量名　=　初始值;

```
int score = 60;
```

也可以先声明，再赋值

声明：　类型　变量名;
赋值：　变量名　=　值;

```
int score;
score = 60;
System.out.println(score);
```

赋值时不用再写类型。

整数类型：能保存的范围不同

byte、short、int 和 long 都能保存整数。

它们占用的位数不同，能保存的整数范围也不同：

类型	位数	取值范围	声明并赋值
byte	8	$-128 \sim 127$	`byte level = 10;`
short	16	$-32768 \sim 32767$	`short count = 1000;`
int	32	$-2^{31} \sim 2^{31}-1$	`int score = 60;`
long	64	$-2^{63} \sim 2^{63}-1$	`long total = 3000000000L;`

- 一般使用 int 即可，范围约为负 21 亿到正 21 亿。
- 需要更大范围时使用 long；上例数字末尾的 L 表示 long。

浮点类型：保存带小数的数值

`float` 和 `double` 的取值范围与精度不同。

类型	位数	有限值范围（约）	有效数字
`float`	32	$-3.4 \times 10^{38} \sim 3.4 \times 10^{38}$	约 7 位
`double`	64	$-1.8 \times 10^{308} \sim 1.8 \times 10^{308}$	约 15–16 位

```
float temperature = 36.5F;
double height = 1.75;
```

- 一般使用 `double`；带小数点的数值默认是 `double`。
- 给 `float` 赋值时，上例在数字末尾加 `F`。
- 浮点数通常是近似值，不能精确表示范围内的每一个小数。

布尔类型：只有 true 和 false

boolean 只有两个值：true（真）和 false（假）。

```
boolean ready = true;
System.out.println(ready);

ready = false;
System.out.println(ready);
```

两次输出依次为 true 和 false，各占一行。

先记住它的写法，具体作用留到后面介绍。

字符类型：用单引号表示一个字符

char 可以保存一个英文字母，比如 A。

```
char letter = 'A';
System.out.println(letter);

letter = 'B';
System.out.println(letter);
```

- 字符使用**单引号**，如 'A'；不要写成双引号 "A"。
- 两次输出依次为 A 和 B，引号本身不会显示。

自动转换与强制转换

自动转换：如 int 转为 long 或 double，Java 会自动完成。

```
int count = 10;
double value = count;
System.out.println(value); // 10.0
```

强制转换：浮点数转整数，或整数变量转为范围更小的类型时，需要显式写出（目标类型）。

```
double price = 19.9;
int whole = (int) price;
System.out.println(whole); // 19
```

- 上例转为整数时舍去小数部分，不是四舍五入。
- 强制转换可能丢失信息；自动转换也不一定保留全部精度。

算术运算符：加、减、乘、除与取余

我们可以对数字做运算，把结果保存到变量中，或直接打印出来。

符号	运算	表达式示例	结果
+	加法	7 + 2	9
-	减法	7 - 2	5
*	乘法	7 * 2	14
/	除法	7 / 2	3
%	取余	7 % 2	1

```
int sum = 7 + 2;
System.out.println(sum);   // 9
```

% 求除法的余数，常被称为“取模”；这里先看正整数。

括号可以改变运算顺序

默认先计算乘法、除法和取余，再计算加法、减法。

没有括号：先乘，再加

```
int result = 2 + 3 * 4;
System.out.println(result); // 14
```

先算 3 * 4 得到 12，再算 2 + 12 得到 14。

加上括号：先计算括号里面的部分

```
result = (2 + 3) * 4;
System.out.println(result); // 20
```

先算 2 + 3 得到 5，再算 5 * 4 得到 20。

超出整数范围：结果可能变成负数

byte 的范围是 $-128 \sim 127$，无法保存 128。

```
byte number = 127;
System.out.println(number + 1); // 128
```

- byte 参与这里的加法时，先提升为 int，得到 128。
- 强制转回 byte 时只保留低 8 位，结果变为 -128。
- 转换不会自动检查范围或报错；应选择范围足够大的类型。

short 对应的边界是 32767：加 1 后转回 short 得到 -32768。

整数除法：先转换，再相除

两个整数相除，结果仍是整数，小数部分向零截断。

```
int total = 7;
int count = 2;
System.out.println(total / count);          // 3
System.out.println((double) total / count); // 3.5
```

想保留小数，先把其中一个操作数转成 double，再做除法。

只把结果赋给 double，或除完再转换，都来不及：

```
double result = total / count;
System.out.println(result);                 // 3.0
System.out.println((double) (total / count)); // 3.0
```

自增、自减与运算后赋值

自增与自减：++ 让变量加 1，-- 让变量减 1。

```
int count = 5;
count++; // count is now 6
count--; // count is now 5
```

复合赋值：把运算结果保存回原变量。以下以 int 变量为例：

简写	对应写法
`count += 2;`	`count = count + 2;`
`count -= 2;`	`count = count - 2;`
`count *= 2;`	`count = count * 2;`
`count /= 2;`	`count = count / 2;`
`count %= 2;`	`count = count % 2;`

比较运算：结果是 boolean

比较两个数值，结果是 `true` 或 `false`，可以保存到 `boolean` 变量中。

运算符	含义	示例	结果
`==`	等于	`7 == 2`	`false`
`!=`	不等于	`7 != 2`	`true`
`>`	大于	`7 > 2`	`true`
`<`	小于	`7 < 2`	`false`
`>=`	大于或等于	`7 >= 7`	`true`
`<=`	小于或等于	`7 <= 2`	`false`

```
boolean passed = 80 >= 60;
System.out.println(passed); // true
```

注意：`=` 是赋值，`==` 才是判断是否相等。

逻辑运算：组合或取反条件

逻辑运算的操作数和结果都是 boolean。

运算符	含义	结果为 true 的条件
&&	与	两边都为 true
||	或	至少一边为 true
!	非（取反）	原来的值为 false

```
int score = 80;
boolean inRange = (score >= 0) && (score <= 100); // true
boolean outside = (score < 0) || (score > 100);   // false
boolean failed = !(score >= 60);                  // false
```

字符串：保存文本信息

字符串可以理解为一串 char，用来保存文本信息。

使用 String 声明变量，用**双引号**包住文字：

```
String name = "Alice";
name = "Bob";
System.out.println(name); // Bob
```

String 的首字母要大写，它不属于八种基本数据类型。

有无双引号，含义不同：

```
int age = 18;
System.out.println(age);   // 18
System.out.println("age"); // age
```

age 读取变量的值；"age" 表示文字本身。

用 + 拼接字符串

+ 可以把字符串连接起来；数值参与拼接时会转换为文本。

```
String name = "Alice";
int age = 18;
String message = name + " is " + age + " years old.";
System.out.println(message); // Alice is 18 years old.
```

注意区分数值加法和字符串拼接：

```
System.out.println(1 + 2);           // 3
System.out.println("Total: " + 1 + 2);   // Total: 12
System.out.println("Total: " + (1 + 2)); // Total: 3
```

- 连续的 + 从左向右组合，遇到字符串就进行拼接。
- 想先算出数值结果，再拼接文字，就把计算部分放在括号中。

综合示例：商品总价计算需求

购买 A、B 两种商品，根据各自的单价和数量计算总价。

- 用 `priceA`、`priceB` 保存两种商品的单价，单位为元。
- 用 `quantityA`、`quantityB` 保存各自的购买数量。
- 直接修改代码中四个变量的初始值。
- 计算总价，并在控制台打印结果。

计算公式：总价 = A 的单价 $\times$ 数量 + B 的单价 $\times$ 数量。

示例数据：A 每件 12.5 元，买 2 件；B 每件 8 元，买 3 件。

综合示例：商品总价计算实现

```
public class Main {
    public static void main(String[] args) {
        double priceA = 12.5;
        double priceB = 8.0;
        int quantityA = 2;
        int quantityB = 3;
        double total = priceA * quantityA + priceB * quantityB;
        System.out.println("Total: " + total);
    }
}
```

- 先分别计算两种商品的金额，再相加得到总价。
- 本例输出 `Total: 49.0`，即总价为 49 元。

新需求：满足条件时才打折

现在增加一个需求：总价**超过 40 元**时打八折。
也就是：当 `total > 40` 时，让 `total` 乘以 0.8；否则保持不变。

if：满足条件时才执行

if 语法：条件为 true 时执行，否则跳过。

```
if（条件）{
    条件成立时执行的语句
}
```

条件必须是 boolean。例如：总价超过 40 元时打八折。

```
double total = 49.0;
if (total > 40) {
    total = total * 0.8;
}
System.out.println("Total: " + total); // Total: 39.2
```

新需求：打折与未打折时输出不同文字

如果打折了，输出文字也要变化：

- 打折时，输出 `Discounted total:` 和折后总价。
- 未打折时，输出 `Total:` 和正常总价。

原来的写法只在 `if` 中修改价格，最后总会执行外面的同一条输出语句，因而无法区分这两种文字。

自然的想法是：**如果打折了，就按折扣文本打印；否则，按正常文本打印。**

用 `else` 表示“否则”，让程序在两条分支中选择一条执行。

if-else：在两条分支中选择一条

条件为 `true` 时执行 `if` 分支，为 `false` 时执行 `else` 分支。

语法

```
if (条件) {
    条件成立时执行
} else {
    条件不成立时执行
}
```

折扣示例

```
double total = 49.0;
if (total > 40) {
    total = total * 0.8;
    System.out.println(
        "Discounted total: " + total);
} else {
    System.out.println(
        "Total: " + total);
}
```

新需求：按总价划分价格档次

假设总价非负，我们希望根据金额输出对应的价格档次。

总价范围	档次
$0 \leq \texttt{total} \leq 40$	便宜（Cheap）
$40 < \texttt{total} \leq 100$	适中（Moderate）
$\texttt{total} > 100$	昂贵（Expensive）

40 元属于“便宜”，100 元属于“适中”，每个总价只属于一个档次。

先用三个独立的 if 实现

```
double total = 80.0;
if (total <= 40) {
    System.out.println("Cheap");
}
if (total > 40 && total <= 100) {
    System.out.println("Moderate");
}
if (total > 100) {
    System.out.println("Expensive");
}
```

这三个判断在逻辑上是一组：对**同一个不变的总价**，恰好只有一个条件成立。

但三个独立的 `if` 仍会从上到下，逐个判断条件。

修改总价后，后面的 if 可能再次成立

假设超过 100 元打五折，并把这个判断放在另外两个判断之间：

```
double total = 180.0;
if (total <= 40) {
    System.out.println("Cheap");
}
if (total > 100) {
    total = total * 0.5;
    System.out.println("Discounted");
}
if (total > 40 && total <= 100) {
    System.out.println("Moderate");
}
```

180 元变成 90 元后，又满足“适中”条件，两个分支都执行了。

else if：一组条件，只执行一个分支

我们需要把多个判断连成一组：**一旦某个条件成立，后面的条件就不再判断。**

```
if (条件 1) {
    条件 1 成立时执行的语句
} else if (条件 2) {
    条件 2 成立时执行的语句
} else {
    以上条件都不成立时执行的语句
}
```

- 从上到下判断，只执行**第一个成立条件**对应的分支。
- 即使分支中修改了变量，也不会继续判断后面的 `else if`。
- 最后的 `else` 可以省略；省略后可能一个分支也不执行。

新需求：余额不足时充值

商品总价 180 元，余额只有 50 元。假设每次充值增加 50 元。

先用 `if`：余额不足，就充值一次。

```
double total = 180.0;
double balance = 50.0;
if (balance < total) {
    balance = balance + 50;
}
System.out.println("Balance: " + balance); // 100.0
```

问题：充值一次后，余额仍然不够付款。

`if` 只判断一次，不会自动回头检查余额是否已经足够。

充值后重新检查，直到余额足够

我们需要的是：**只要余额不足，就继续充值。**

1. 检查余额是否小于商品总价。
2. 如果不足，充值 50 元，然后回到第一步。
3. 如果已经足够，结束充值，继续付款。

本例余额变化：$50 \to 100 \to 150 \to 200$ 元。

余额达到 200 元时，不再满足“余额不足”，于是停止充值。

用 `while` 表示这种**条件成立就反复执行**的过程。

while：条件成立就重复执行

每轮先判断条件：为 `true` 时执行循环体，再回到条件处；为 `false` 时退出。

语法

```
while（条件）{
    反复执行的语句
}
```

条件必须是 `boolean`。

充值与付款

```
double total = 180.0;
double balance = 50.0;
while (balance < total) {
    balance = balance + 50;
}
balance = balance - total;
System.out.println(
    "Remaining: " + balance); // 20.0
```

- 一开始余额就足够时，循环一次也不执行，直接付款。
- 如果余额不足，却不在循环中增加余额，就会一直循环。

用 while 计数：打印商品编号

新需求：依次打印第 1～5 件商品的编号。

```
int number = 1;
while (number <= 5) {
    System.out.println("Item: " + number);
    number++;
}
```

- 初始化：`int number = 1;`，从 1 开始。
- 继续条件：`number <= 5`，编号不超过 5。
- 更新：`number++`，每轮结束后加 1。

三个部分分散在不同位置；如果忘记 `number++`，就会一直打印编号 1。

for：把计数循环的三个部分写在一起

for 把初始化、继续条件和更新集中到循环开头，便于检查。

```
for（初始化；继续条件；更新）{
    每轮执行的语句
}
```

同样打印第 1～5 件商品的编号：

```
for (int number = 1; number <= 5; number++) {
    System.out.println("Item: " + number);
}
```

- 初始化只执行一次；每轮先判断条件，再执行循环体。
- 循环体结束后执行更新，再回到条件处；条件为假时结束。
- for 仍需写更新语句，并不会自动帮我们加 1。

switch：根据一个值选择分支

当多个分支都在判断**同一个表达式等于哪个值**时，可以使用 switch。

```
switch（表达式）{
    case 常量值 1:
        对应的语句
        break;
    case 常量值 2:
        对应的语句
        break;
    default:
        没有匹配值时执行的语句
}
```

- case 指定匹配值，不能写成 case total > 40:。
- break 结束当前 switch；default 可以省略。

switch 示例：根据编号选择配送方式

```
int delivery = 2;
switch (delivery) {
    case 1:
        System.out.println("Pickup");
        break;
    case 2:
        System.out.println("Express");
        break;
    default:
        System.out.println("Unknown delivery");
}
```

本例输出 Express。**若省略 break**，会贯穿执行后续分支语句，不再检查后面的 case 是否匹配。

do-while：先执行，再判断条件

while 先判断条件；do--while 先执行一次循环体，再判断是否继续。

```
do {
    反复执行的语句
} while (条件);
```

- 先执行循环体，再计算 boolean 条件。
- 条件为 true 时回到循环体，为 false 时结束。
- 循环体**至少执行一次**，适合“先做一次，再决定是否重复”。
- 注意末尾的**分号**：while (条件);。

do-while 示例：至少打印一次编号

假设要求先打印一个编号，再判断是否需要继续。

```
int number = 1;
int limit = 3;
do {
    System.out.println("Item: " + number);
    number++;
} while (number <= limit);
```

本例依次打印编号 1、2、3。

如果把 `limit` 改为 0：

- `do--while` 仍然会打印编号 1，然后才判断条件并结束。
- 若改用 `while (number <= limit)`，循环一次也不会执行。

练习：打印 1000 以内能被 7 整除的数

需求：打印 1～1000 中所有能被 7 整除的整数，每个数占一行。

思路：

1. 用 `for` 循环，依次检查 1 到 1000。
2. 用 `number % 7 == 0` 判断除以 7 的余数是否为 0。
3. 条件成立就打印，否则继续检查下一个数。

预期输出：7、14、21、……、994。

这里需要把 `if` 放在 `for` 的循环体内，让每个数都经过判断。

实现：在循环中判断是否能被 7 整除

```
public class Main {
    public static void main(String[] args) {
        for (int number = 1; number <= 1000; number++) {
            if (number % 7 == 0) {
                System.out.println(number);
            }
        }
    }
}
```

- 外层 for 控制依次检查哪些数字。
- 内层 if 决定当前数字是否需要打印。
- 不论是否打印，本轮结束后都会执行 number++。

更新需求：一共打印了多少个数

除了打印 1～1000 中能被 7 整除的数，我们还想知道**一共有多少个**。

1. 在循环**外面**声明计数变量 `count`，初始化为 0。
2. 找到符合条件的数时，打印这个数，并执行 `count++`。
3. 循环结束后，打印 `count`，得到总数。

为什么放在循环外？

计数需要保留前面各轮的结果。如果每轮都执行 `int count = 0;`，就会反复从 0 开始，无法累计。

实现：打印数字并累计个数

```java
public class Main {
    public static void main(String[] args) {
        int count = 0;
        for (int number = 1; number <= 1000; number++) {
            if (number % 7 == 0) {
                System.out.println(number);
                count++;
            }
        }
        System.out.println("Count: " + count);
    }
}
```

count++ 只在打印时执行；最后输出 Count：142。